

Towards Personal Continual Learning in Large Language Models:

Asymmetric Sphere Architecture and Background Micro Fine-Tuning

Author: Paras Lashkari

Affiliation: Independent AI Researcher

Email: parashlashkari@gmail.com

Date: May 2026

Type: Position Paper / Research Proposal

Status: Open for community research and validation

Experimental Status: Early-stage experiments currently in progress. Preliminary results expected soon. This paper will be updated with experimental findings upon completion.

Field: Continual Learning, Large Language Models, Personal AI

Contact: For questions, collaboration, or validation of these ideas, contact Paras Lashkari at parashlashkari@gmail.com. The author welcomes feedback from researchers working in continual learning, personal AI, and LLM architecture.

Abstract

We propose two novel architectural approaches for enabling continual learning in Large Language Models (LLMs) for personal AI assistants, without expensive GPU retraining or catastrophic forgetting. The first approach, **Internal KV-Sphere Architecture**, attaches new knowledge as Key-Value (KV) token pairs directly inside a secondary frozen model, anchored to similar existing vectors via attention similarity. The second approach, **Background Micro Fine-Tuning (BMFT)**, uses a small fine-tuner running continuously on CPU to incrementally update a secondary model from a personal interaction buffer, while the primary model remains frozen. Both approaches are designed for zero-GPU, always-learning, personal AI systems. We present the motivations, proposed architectures, known challenges, predicted experimental outcomes, and open problems for community research.

1. Introduction

1.1 The Problem

Large Language Models (LLMs) are frozen at training time. After deployment, they cannot learn new information without:

1. **Full retraining** - requires massive GPU clusters, millions in compute cost

2. **Fine-tuning** - risks catastrophic forgetting of original knowledge
3. **RAG (Retrieval Augmented Generation)** - knowledge stays external, never truly internalized
4. **LoRA/Adapters** - small capacity, limited knowledge integration

For personal AI assistants that need to learn daily from user interactions—new facts, preferences, habits, corrections—none of these solutions is practical.

1.2 Our Motivation

A personal AI assistant like NATSU should:

- Learn that "bhai" means the user's brother (contact mapping)
- Remember the user switched to a vegetarian diet last Tuesday
- Know the user's new job title changed this month
- Understand new tools the user started using

None of this should require retraining. All of this should happen automatically, continuously, and cheaply.

1.3 Our Contribution

We propose two architectures targeting this exact problem:

1. **Internal KV-Sphere Architecture (IKSA)** - new knowledge stored as KV tokens inside a secondary frozen sphere, attached to semantically similar vectors
2. **Background Micro Fine-Tuning (BMFT)** - a small fine-tuner runs on CPU overnight, updating a secondary sphere from daily interaction memory

Both share a core principle:

Primary model = frozen forever. Secondary model = absorbs new knowledge. User experience = uninterrupted.

2. Related Work

2.1 What Already Exists

Approach	Key Papers	Limitation
Continual Fine-Tuning	Kirkpatrick et al. 2017 (EWC)	Catastrophic forgetting
LoRA / Adapters	Hu et al. 2022	Small capacity, static
RAG	Lewis et al. 2020	External, not internalized
MoE	Shazeer et al. 2017	Trained jointly, not dynamic
Model Editing (ROME)	Meng et al. 2022	Surgical, not continuous
MELT	Conchello et al. 2026	Multi-step reasoning, not continual learning
SplitFrozen	Ma et al. 2025	Server-side fine-tuning, not personal
Brainstacks	Abu Ayyash 2026	Frozen adapter stacks, not dynamic KV
MemGPT	Packer et al. 2023	External memory only

2.2 What is Missing

No existing work addresses:

- **Asymmetric frozen architecture** (large frozen + small updateable) for continual learning
- **Internal KV attachment** to semantically similar vectors in secondary frozen model
- **CPU-only background micro fine-tuning** for personal daily knowledge updates
- **Personal AI use case** with zero GPU, zero interruption, daily learning

3. Architecture 1: Internal KV-Sphere Architecture (IKSA)

3.1 Core Idea

Instead of replacing vectors or storing knowledge externally, new knowledge is encoded as **KV token pairs** and attached **inside** a secondary frozen model, anchored to semantically similar existing vectors.

```

PRIMARY SPHERE A (Frozen)
Large LLM (e.g. 70B params)
General world knowledge
NEVER modified
    |
    | Attention Bridge
    | (similarity-based routing)
    v
SECONDARY SPHERE B (Frozen base) |
Smaller LLM (e.g. 33M params)
    |
Original Vectors:
[v1] [v2] [v3] [v4] [v5]...
    |
+ Attached KV Pairs (new knowledge):
v3 -> [(k_new1, val_new1),
      (k_new2, val_new2)]
    |
v7 -> [(k_new3, val_new3)]
    |
KV pairs attached to most similar
existing vector via cosine similarity |

```

3.2 How New Knowledge is Added

Step 1: Encode new knowledge

```

new_info = "Python 4.0 was released in March 2026"
new_token_embedding = tokenizer.encode_to_embedding(new_info)
# Result: dense vector representing this knowledge

```

Step 2: Find most similar existing vector in Sphere B

```

def find_anchor(new_embedding, sphere_B):
    similarities = cosine_similarity(new_embedding, sphere_B.all_vectors)
    anchor_id = argmax(similarities)
    return anchor_id
# Example: "Python 4.0" most similar to existing Python/programming vectors

```

Step 3: Attach KV pair to anchor vector

```

sphere_B.kv_attachments[anchor_id].append({
    'key': new_token_embedding,
    'value': new_token_embedding,
    'timestamp': now(),
    'importance': calculate_importance(new_info),
    'access_count': 0
})

```

Step 4: During inference extended attention

```
def extended_attention(query, sphere_B):
    # Standard attention on frozen vectors
    base_output = sphere_B.standard_attention(query)
    # Additional attention on KV attachments
    for vector_id, kv_pairs in sphere_B.kv_attachments.items():
        base_vector_similarity = cosine_similarity(query, vector_id)
        if base_vector_similarity > THRESHOLD:
            # This anchor is relevant check its attachments
            for k, v in kv_pairs:
                attachment_score = attention_score(query, k)
                base_output += attachment_score * v
    return normalize(base_output)
```

3.3 Multi-Sphere Extension

Multiple secondary spheres can be attached to the primary for domain specialization:

Primary Sphere A (70B, frozen)

- Sphere B: Personal knowledge (user habits, preferences)
- Sphere C: Domain knowledge (coding, work)
- Sphere D: Current events (news, recent facts)
- Sphere E: Skills knowledge (apps, tools learned)

Each sphere:

- Has its own KV attachment store
- Specializes in one domain
- Can be reset/replaced without affecting others
- Same or smaller size than primary

3.4 Predicted Properties

Property	Prediction	Reasoning
Primary forgetting rate	0%	Primary sphere frozen by design
KV retrieval accuracy	70-85%	Similarity-based attachment is semantically meaningful
Inference overhead	+15-30%	Additional attention computation per sphere
Knowledge capacity per sphere	~100K-1M facts	Depends on embedding dimension and attachment limits
Latency per query	+20-50ms	Linear in number of active spheres

4. Architecture 2: Background Micro Fine-Tuning (BMFT)

4.1 Core Idea

A small fine-tuner runs **continuously in the background on CPU**, pulling new knowledge from a daily interaction memory buffer and incrementally fine-tuning a secondary model. The primary model is never touched.

```
PRIMARY SPHERE A
Frozen forever
Never changes
    |
SECONDARY SPHERE B
Updated by fine-tuner |
Pre-allocated empty space |
for new knowledge
    |
    ^ Fine-tuning updates
BACKGROUND FINE-TUNER |
Runs on CPU
Small + efficient
LoRA-style updates
Processes 1K-10K tokens |
per cycle
    ^ Pulls from
MEMORY BUFFER
Daily interactions
User corrections
New facts learned
Filtered for importance |
```

4.2 System Components

Component 1: Memory Buffer

```
class PersonalMemoryBuffer:
    def __init__(self, max_tokens=50000):
        self.buffer = []
        self.max_tokens = max_tokens

    def add_interaction(self, conversation):
        # Extract learnable knowledge
        knowledge = self.extract_knowledge(conversation)
        if knowledge:
            self.buffer.append({
                'tokens': tokenize(knowledge),
                'timestamp': now(),
                'importance': self.score_importance(knowledge),
                'type': classify_type(knowledge)
            })
            # Types: fact, preference, correction, skill, contact

    def extract_knowledge(self, conversation):
        # Filter what is worth learning:
        # New facts user stated
        # Corrections to AI mistakes
        # - User preferences expressed
        # - Contact/relationship info
        # - New skills or tools mentioned
        # NOT learned:
        # Casual conversation
        # - Questions (not answers)
        # Repeated known information
        pass
```

Component 2: Background Fine-Tuner

```
class BackgroundMicroFineTuner:
    def __init__(self, sphere_B, cpu_only=True):
        self.sphere_B = sphere_B
        self.device = 'cpu' # Always CPU
        self.is_running = False
        self.schedule = 'idle' # Run when user idle

    def should_trigger(self, buffer):
        # Trigger conditions
        return (
            len(buffer.tokens) > 1000 and
            not self.is_running and
            self.user_is_idle() and
            self.time_is_appropriate()
            # Enough new data
            # Not already running
            # User not active
            # e.g. 2-4 AM
        )

    def run_micro_finetune(self, batch):
        # Fine-tune Sphere B on small batch
        # LoRA-style: only update small subset of parameters
        self.is_running = True
        # Freeze most of Sphere B
        # Only update LoRA adapter layers
        lora_params = self.sphere_B.get_lora_params()
        optimizer = AdamW(lora_params, lr=1e-5)
        for step in range(MAX_STEPS):
            loss = self.compute_loss(batch)
            loss.backward()
            optimizer.step()
            optimizer.zero_grad()
            # Safety check: if loss spikes, stop
            if loss > SAFETY_THRESHOLD:
                self.rollback()
                break
        self.sphere_B.save_checkpoint()
        self.is_running = False

    def user_is_idle(self):
        # Check if user has not interacted in X minutes
        return time_since_last_interaction() > 300 # 5 minutes
```


Component 3: Pre-Allocated Expansion Space

```
class SphereB_WithExpansion:
    def __init__(self, base_model, expansion_regions=10):
        self.base = load_model(base_model)
        # Pre-allocate empty expansion regions
        self.expansion_regions = [
            ExpansionRegion(size=1_000_000) # 1M params each
            for _ in range(expansion_regions)
        ]
        # All initialized to zeros (inactive)
        # Fine-tuner activates them as needed
```

4.3 Training Schedule

DAILY CYCLE:

06:00 AM — User wakes up, starts using NATSU (Memory buffer collecting interactions)

During day — Buffer fills with new knowledge:

- "My meeting is at 3 PM" (preference/fact)
- "I prefer dark mode always" (preference)
- Corrections to wrong answers

11:00 PM — User goes to sleep. Buffer has 500-5000 new tokens

02:00 AM — Fine-tuner triggers:

- User idle confirmed (5+ hours)
- Buffer size threshold met
- System resources available

02:00-04:00 AM — CPU fine-tuning runs:

- LoRA updates to Sphere B
- ~1-3 epochs on buffer content
- Loss monitored for safety. Rollback if catastrophic

06:00 AM — User wakes up:

- Sphere B has yesterday's knowledge
- Buffer cleared (processed tokens)
- Ready for new day

4.4 Predicted Properties

Property	Prediction	Reasoning
Primary forgetting rate	0%	Primary sphere frozen by design
Secondary forgetting rate	5-15%	LoRA updates mitigate but don't eliminate
CPU compute time (1K tokens)	30-120 minutes	Small batch, small model, CPU-only
CPU compute time (10K tokens)	4-12 hours	Overnight feasibility window
Knowledge retention in B	70-90%	LoRA + small LR + few epochs
User experience interruption	0%	Background process, user asleep
New knowledge accuracy next day	75-90%	Small dataset, focused fine-tuning

5. Hybrid Architecture: IKSA + BMFT Combined

We propose combining both approaches for optimal performance:

IMMEDIATE LEARNING (IKSA):

New info arrives → KV attachment to Sphere B (instant). User can use new knowledge immediately.

DEEP LEARNING (BMFT):

Overnight → Fine-tuner processes buffer. Important facts get "baked into" Sphere B weights. KV attachments cleared after consolidation.

Result:

- Instant availability (IKSA handles day-1)
- True internalization (BMFT handles long-term)
- No GPU needed
- Primary never changes
- Infinite expansion via new spheres

5.1 Hybrid Flow

```
class HybridContinualLearningSystem:
    def __init__(self):
        self.sphere_A = FrozenPrimaryModel() # Never changes
        self.sphere_B = SphereWithKVAndLoRA() # Dual capability
        self.memory_buffer = PersonalMemoryBuffer()
        self.fine_tuner = BackgroundMicroFineTuner(self.sphere_B)
        self.kv_manager = InternalKVManager(self.sphere_B)

    def process_new_knowledge(self, info):
        # IMMEDIATE: Add to KV store (instant)
        self.kv_manager.attach(info)
        # BUFFERED: Add to fine-tuning buffer
        self.memory_buffer.add(info)
        # BACKGROUND: Fine-tuner picks up buffer overnight (automatic, scheduled)

    def query(self, question):
        # Query primary (always reliable)
        primary_answer = self.sphere_A.forward(question)
        # Query secondary (KV + fine-tuned weights)
        secondary_answer = self.sphere_B.extended_attention(question)
        # Combine with recency bias
        return combine(primary_answer, secondary_answer, recency_weight=0.7)
```

6. Problems We Face

6.1 Solved Problems (Proposed Solutions Included)

Problem 1: Attention Routing Between Spheres

Issue: How does attention know to check Sphere B AND Sphere A for every query?

Proposed Solution:

Option A: Always-On (Simple)

- Every query checks all spheres
- Results weighted by similarity score
- Con: Slow for many spheres

Option B: Router Network (Smart)

- Small classifier predicts relevant spheres
- Only relevant spheres queried
- Con: Router needs training

Option C: Similarity Threshold (Balanced)

- Query embedding compared to sphere "summary embedding"

- Only spheres with similarity > 0.4 queried
- Predicted best balance of speed + coverage

Recommended Starting Point: Option A for ≤5 spheres, Option C for >5 spheres

Problem 2: KV Capacity Limits

Issue: Each sphere has finite capacity for KV attachments

Proposed Solution:

```
def manage_kv_capacity(sphere, max_attachments=10000):
    if len(sphere.kv_attachments) > max_attachments:
        # Strategy 1: Remove least recently used
        lru_key = get_least_recently_used(sphere.kv_attachments)
        sphere.kv_attachments.remove(lru_key)
        # Strategy 2: Create new sphere for overflow
        if no_space_after_lru:
            new_sphere = create_new_sphere()
            connect_to_primary(new_sphere)
```

Predicted optimal: LRU eviction + new sphere creation at 80% capacity

Problem 3: Conflicting Information

Issue: Sphere A says "Python latest = 3.9", KV says "Python latest = 4.0"

Proposed Solution:

```
def resolve_conflict(answer_A, answer_B, query):
    # Recency wins for factual/temporal queries
    if is_temporal_query(query):
        return max_by_timestamp(answer_A, answer_B)
    # Confidence wins for factual queries
    if is_factual_query(query):
        return max_by_confidence(answer_A, answer_B)
    # Both presented for subjective queries
    return merge_answers(answer_A, answer_B)
```

Predicted accuracy: 80-90% correct conflict resolution with this rule

Problem 4: Catastrophic Forgetting in Sphere B (BMFT)

Issue: Fine-tuning Sphere B risks overwriting existing knowledge

Proposed Solution:

```

# Use LoRA (Low-Rank Adaptation) style updates
# Only small matrices updated, core weights frozen
class SafeFineTuner:
    def finetune_safely(self, sphere_B, batch):
        # Freeze all core weights
        for param in sphere_B.core_params:
            param.requires_grad = False
        # Only train LoRA adapters (tiny)
        for param in sphere_B.lora_params:
            param.requires_grad = True
        # Use very small learning rate
        optimizer = AdamW(lr=1e-6) # 10x smaller than normal
        # Test on held-out old knowledge after each epoch
        # If forgetting > 5%, rollback immediately
        if forgetting_rate > 0.05:
            rollback_to_checkpoint()

```

Predicted forgetting rate with this approach: 2-8% (acceptable for personal use)

Problem 5: CPU Speed (Is it Fast Enough?)

Issue: CPU fine-tuning is slow - will it finish overnight?

Proposed Calculation:

Parameters in Sphere B (33M model): 33,000,000

LoRA rank 8 adapter: ~200,000 trainable params (0.6% of model)

Average CPU throughput: ~10M operations/second

Tokens per batch: 1,000

Training steps: 100

Estimated time:

= (200,000 params * 100 steps * 1000 tokens) / 10M ops/sec

= ~2,000 seconds ≈ 33 minutes

For 10,000 tokens:

= ~5.5 hours

fits overnight window

Predicted verdict: BMFT is feasible on CPU for personal daily use

6.2 Partially Solved Problems

Problem 6: What to Extract from Conversations (Knowledge Filtering)

Issue: Not every conversation contains learnable knowledge. How to filter?

Partial Solution:

```

LEARN_THESE = [
    "user stated a fact about themselves",
    "user corrected AI mistake",
    "user expressed preference",
    "user mentioned contact/relationship",
    "user described new skill or tool"
]
DONT_LEARN = [
    "casual greeting",
    "question without answer",
    "AI hallucination",
    "repeated known info",
    "sensitive personal data"
]

```

Unsolved part: Automated classification of conversations into these categories requires its own small classifier. Accuracy of this classifier directly affects knowledge quality.

Still needs research: Optimal knowledge extraction pipeline for personal AI

Problem 7: Sphere Drift Over Time

Issue: After months of fine-tuning, Sphere B diverges from Sphere A. Attention paths weaken.

Partial Solution:

```

def monitor_drift(sphere_A, sphere_B):
    similarity = cosine_similarity(
        sphere_A.mean_embedding(),
        sphere_B.mean_embedding()
    )
    if similarity < 0.5: # Too different
        # Option A: Reset Sphere B to copy of A
        # (loses accumulated knowledge)
        # Option B: Distill Sphere B into new Sphere C
        # (preserves knowledge, creates fresh sphere)
        distill(sphere_B, new_sphere_C)

```

Unsolved part: Optimal drift threshold and recovery strategy unknown. Needs empirical study.

6.3 Unsolved Problems (Open Research Questions)

These problems do not yet have clear solutions. We present them for community research:

Open Problem 1: Optimal KV Anchor Selection

Question: When attaching new knowledge as KV to existing vector, which existing vector should be the anchor?

Options tried:

- Most similar vector (cosine similarity) → May create crowded anchors
- Random selection → Poor retrieval
- Importance-weighted selection → How to measure importance?

Why unsolved: No empirical study on which strategy maximizes retrieval accuracy in this specific architecture. Requires systematic ablation study on multiple datasets.

Open Problem 2: Multi-Sphere Attention Integration

Question: When querying multiple spheres simultaneously, how should results be combined?

Attempts:

- Simple average → Loses specificity
- Weighted by similarity score → Weights need calibration
- Learned combiner → Requires training

Why unsolved: Optimal combination function depends on query type, knowledge distribution across spheres, and relative sphere sizes. No theoretical framework exists for this asymmetric architecture.

Open Problem 3: Long-Term Stability of BMFT

Question: Does daily fine-tuning over 1 year cause Sphere B to become unstable or incoherent?

Concern: 365 daily fine-tuning cycles on Sphere B, each modifying LoRA adapters, may lead to:

- Conflicting updates (today's knowledge contradicts last month's)
- Adapter saturation (LoRA adapters run out of capacity)
- Compounding errors (small mistakes accumulate)

Why unsolved: No long-term empirical study exists on daily micro fine-tuning over extended periods. Requires longitudinal experiment of 6-12 months.

Open Problem 4: Attention Head Compatibility Across Spheres

Question: When Sphere B is a different size than Sphere A (e.g., 33M vs 70B), how do attention heads communicate?

Problem:

- Sphere A attention heads have dimension $d = 4096$
- Sphere B attention heads have dimension $d = 512$

These cannot directly communicate without projection

Partial attempt:

```
# Projection layer bridges dimension mismatch
projection = nn.Linear(512, 4096) # Sphere B -> Sphere A space
projected_output = projection(sphere_B_output)
combined = attention(sphere_A_query, projected_output)
```

Why still open: Projection layers need training. What data do you train them on? How do you ensure semantic alignment after projection? This is an unsolved architectural question.

Open Problem 5: Privacy and Security of Personal Knowledge

Question: The memory buffer contains sensitive personal information. How to protect it?

Concerns:

- Buffer stored on device - vulnerable to data theft
- Fine-tuned Sphere B "contains" user information — can it be extracted?
- Model inversion attacks may recover personal data from fine-tuned weights

Why unsolved: Personal AI systems that fine-tune on private data introduce new privacy attack surfaces not yet studied systematically in this architecture.

7. Predicted Experimental Results

Note: These are theoretical predictions based on first-principles reasoning. Empirical validation is needed.

7.1 IKSA Predicted Results

Experiment 1: Knowledge Retrieval Accuracy

Setup:

- Sphere A: GPT-2 Medium (355M, frozen)
- Sphere B: GPT-2 Small (117M, frozen base)
- KV injections: 1000 new facts
- Test: Can system retrieve injected facts?

Metric	Predicted Result	Confidence
Retrieval accuracy (direct facts)	72-85%	Medium
Retrieval accuracy (paraphrased queries)	55-70%	Low-Medium
Primary model forgetting	0%	High
Inference latency increase	+25-40ms	Medium
Memory overhead (1000 KV pairs)	~50MB	High

Experiment 2: Multi-Sphere Scaling

Setup: Add 3-5 spheres, each with 1000 KV pairs

Metric	Predicted Result	Confidence
Query latency (3 spheres)	+60-100ms	Medium
Query latency (5 spheres)	+100-180ms	Medium
Coverage improvement vs single sphere	+30-50%	Low
Confusion between spheres	10-20% error increase	Low

7.2 BMFT Predicted Results

Experiment 1: Knowledge Retention After 1 Week

Setup:

- Sphere B: 33M model
- Daily fine-tuning: 1000 tokens/day for 7 days
- Test: Can it recall facts from day 1 after day 7?

Metric	Predicted Result	Confidence
Day-1 facts recalled on day 7	65-80%	Medium
Original Sphere B knowledge retained	85-95%	Medium
CPU time per daily cycle (1K tokens)	20-60 minutes	High
Loss convergence (3 epochs)	Yes	High
Rollback triggered	<5% of cycles	Medium

Experiment 2: Long-Term (30 Days)

Metric	Predicted Result	Confidence
Knowledge from day 1 recalled at day 30	40-65%	Low
LoRA adapter saturation	Not observed (< 30 days)	Low
Sphere drift (cosine sim to original)	0.7-0.9 similarity	Low
Total CPU time over 30 days	10-30 hours	Medium

7.3 Hybrid System Predicted Results

Combining IKSA (short-term) + BMFT (long-term)

Metric	Predicted Result	Confidence
Same-day knowledge availability	90-95%	High
Week-old knowledge availability	80-90%	Medium
Month-old knowledge availability	60-80%	Low
User experience interruption	0%	High
GPU requirement	0%	High
Primary model forgetting	0%	High

8. Comparison with Existing Work

Property	RAG	LoRA	MELT	IKSA (ours)	BMFT (ours)	Hybrid (ours)
No GPU needed	✗	✗	✗	✓	✓	✓
Zero primary forgetting	✓	✗	✗	✓	✓	✓
Knowledge internalized	✗	✓	✓	✓	✓	✓
Instant learning	✓	✗	✗	✓	✗	✓
Infinite expansion	✓	✗	✗	✓	✗	✓
Personal use scale	✗	✗	✗	✓	✓	✓
Daily learning	✓	✗	✗	✓	✓	✓
Multi-domain spheres	✗	✗	✗	✓	✗	✓

9. Implementation Roadmap for Researchers

For researchers wanting to validate these ideas:

Phase 1: Minimal Proof of Concept (2-4 weeks)

Goal: Prove IKSA basic retrieval works

Setup:

```
# Use GPT-2 Small as Sphere B (117M, manageable)
# Inject 100 synthetic facts as KV pairs
# Test retrieval with direct and paraphrased queries
# Measure: accuracy, latency, memory overhead
```

Success criteria:

- KV retrieval accuracy > 70% on direct queries
- No degradation of original Sphere B accuracy
- Latency increase < 100ms

Phase 2: BMFT Validation (4-8 weeks)

Goal: Prove CPU fine-tuning is feasible and effective

Setup:

```
# Use GPT-2 Small (117M) as Sphere B
# Fine-tune on 1000 synthetic personal facts
# Use LoRA rank 4 (minimal parameters)
# Run on CPU only
# Measure: time, accuracy, forgetting rate
```

Success criteria:

- Fine-tuning completes in < 2 hours on standard CPU
- New knowledge accuracy > 75%
- Original knowledge retention > 85%

Phase 3: Long-Term Study (3-6 months)

Goal: Validate stability over extended period

Setup:

```
# Daily micro fine-tuning for 90 days
# 500-1000 tokens per day
# Track: drift, forgetting rate, latency
```

Success criteria:

- System remains stable at 90 days
- Cumulative knowledge recall > 60%
- No catastrophic failures

10. Theoretical Analysis

10.1 Why KV Attachment to Similar Vectors Works (Hypothesis)

In transformer models, semantically similar concepts share nearby regions of the embedding space. This is demonstrated by:

- Word2Vec nearest neighbor properties

- BERT sentence embedding clustering
- GPT attention pattern analysis

Hypothesis: If new knowledge is semantically related to existing knowledge, attaching KV pairs to the nearest existing vector creates a natural retrieval path. When a query activates attention to the anchor vector, it will also find the attached KV pairs.

Prediction: Retrieval accuracy scales with semantic similarity between new knowledge and anchor vector. Highly related facts → high retrieval. Unrelated facts → poor retrieval.

Implication: IKSA works best for knowledge that is semantically connected to existing model knowledge. For completely novel domains, a new dedicated sphere should be created.

10.2 Why CPU Micro Fine-Tuning is Viable (Analysis)

Traditional fine-tuning fails on CPU because:

- Large batch sizes require GPU memory
- Many epochs on large datasets take weeks

Our approach differs:

- **Tiny dataset:** 1000-10000 tokens (vs millions in standard fine-tuning)
- **Few epochs:** 1-3 (vs 10-100 in standard fine-tuning)
- **LoRA:** Only 0.1-1% of parameters updated
- **Low learning rate:** 100-1000x smaller than standard

Mathematical estimate:

Standard fine-tuning: 1B tokens * 100 epochs * all params = days on GPU

BMFT: 1000 tokens * 3 epochs * 0.1% params = hours on CPU

11. Limitations and Honest Assessment

11.1 What We Don't Know

1. **Whether retrieval accuracy is practically useful** - 75% accuracy may not be enough for real personal assistant use
2. **Long-term stability** - We have no empirical data beyond theoretical predictions
3. **Knowledge extraction quality** - If the wrong things are learned, Sphere B becomes noisy
4. **Attention projection across different-sized spheres** - Unsolved architectural challenge
5. **Privacy implications** - Personal knowledge in fine-tuned weights is a new security surface

11.2 What This Paper Is NOT Claiming

- We are **NOT** claiming this is better than existing methods (no experiments yet)
- We are **NOT** claiming this is production-ready

- We are **NOT** claiming all problems are solved
- We ARE claiming this is a **novel direction** worth investigating

12. Conclusion

We have presented two novel architectural approaches for personal continual learning in LLMs:

1. **IKSA** - Internal KV-Sphere Architecture enabling instant knowledge attachment to frozen secondary models via semantic similarity anchoring
2. **BMFT** - Background Micro Fine-Tuning enabling overnight CPU-based knowledge internalization without user interruption

Both share the core innovation of **asymmetric frozen architecture**: a permanent primary sphere combined with an updateable secondary sphere, enabling true continual learning at personal scale without GPU requirements.

We acknowledge multiple open problems - particularly attention routing between asymmetric spheres, long-term stability, and knowledge extraction quality and present these explicitly for community research.

The fundamental insight of this work is:

Controlled forgetting in a disposable secondary model, backed by a frozen primary reference, enables practical personal continual learning at zero GPU cost.

We release this proposal publicly and invite the research community to validate, extend, or disprove these ideas experimentally.

13. Open Problems Summary (For Community)

#	Problem	Difficulty	Status
1	Optimal KV anchor selection strategy	Medium	Open
2	Multi-sphere attention combination	Hard	Open
3	Long-term BMFT stability (1 year)	Hard	Open
4	Cross-dimension attention projection	Hard	Open
5	Privacy/security of personal fine-tuned weights	Medium	Open
6	Knowledge extraction classifier	Medium	Partial
7	Sphere drift recovery strategy	Medium	Partial
8	Optimal LoRA rank for micro fine-tuning	Easy	Partial

14. Experimental Status & Updates

Current Status (May 2026):

- Architecture proposed
- Theoretical analysis complete
- Predictions documented
- Experiments: IN PROGRESS

Coming Soon:

- Preliminary results on 33M parameter model
- KV attachment retrieval accuracy measurements
- CPU fine-tuning time benchmarks
- Initial forgetting rate measurements

To receive updates when results are published:

Contact: parashlashkari@gmail.com

This paper will be versioned. v1.0 = proposal only. v1.1+ = includes experimental results.

```
@techreport{lashkari2026personal,  
  title={Towards Personal Continual Learning in Large Language Models: Asymmetric  
Sphere Architecture and Background Micro Fine-Tuning},  
  author={Lashkari, Paras},  
  year={2026},  
  month={May},  
  type={Position Paper / Research Proposal},  
  institution={Independent AI Research},  
  email={parashlashkari@gmail.com},  
  note={Open research proposal - community validation invited},  
  url={[ArXiv link when posted]}  
}
```

How to cite this work (APA):

Lashkari, P. (2026, May). Towards Personal Continual Learning in Large Language Models: Asymmetric Sphere Architecture and Background Micro Fine-Tuning. Independent AI Research. parashlashkari@gmail.com

How to cite this work (plain text):

Paras Lashkari (2026). "Towards Personal Continual Learning in Large Language Models: Asymmetric Sphere Architecture and Background Micro Fine-Tuning." Independent AI Research. Contact: parashlashkari@gmail.com

Acknowledgments

This research emerged from independent first-principles thinking about the limitations of current LLM architectures for personal AI use cases.

Paras Lashkari is a medical student working independently on AI systems research. No institutional funding or compute resources were used.

Special thanks to the open-source AI community whose published work informed the related work section.

Author contact: parashlashkari@gmail.com

Open to: Collaboration, feedback, experimental validation, co-authorship on follow-up work

END OF PAPER

Word count: ~5,800

Version: 1.0

License: Creative Commons BY 4.0 - Free to use with attribution